Name=Atharv Muik Roll no=27 prn=12412094 batch=CSE(AI)-D-2

Double-click (or enter) to edit

```
import pandas as pd
import matplotlib.pyplot as plt

df=pd.read_csv("/content/house_prices_practice.csv")
df.head()
```

	Id	OverallQual	GrLivArea	GarageCars	TotalBsmtSF	YearBuilt	FullBath	BedroomAbvGr	LotArea	SalePrice
0	1	7	1560	0	1658	1969	2	1	8059	177106
1	2	4	2827	2	1319	2012	3	4	13530	301044
2	3	8	3920	0	841	2010	1	4	9010	360609
3	4	5	3044	0	1058	1998	0	4	13207	240556
4	5	7	801	1	2428	2020	0	1	9117	193656

check dataset info

```
df.shape
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 300 entries, 0 to 299
Data columns (total 10 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Id            300 non-null    int64
 1   OverallQual   300 non-null    int64
 2   GrLivArea     300 non-null    int64
 3   GarageCars    300 non-null    int64
 4   TotalBsmtSF   300 non-null    int64
 5   YearBuilt     300 non-null    int64
 6   FullBath      300 non-null    int64
 7   BedroomAbvGr  300 non-null    int64
 8   LotArea       300 non-null    int64
 9   SalePrice     300 non-null    int64
dtypes: int64(10)
memory usage: 23.6 KB
```

	Id	OverallQual	GrLivArea	GarageCars	TotalBsmtSF	YearBuilt	FullBath	BedroomAbvGr	LotArea	
count	300.000000	300.000000	300.000000	300.000000	300.000000	300.000000	300.000000	300.000000	300.000000	3
mean	150.500000	5.326667	2307.386667	1.330000	1468.796667	1986.163333	1.523333	2.926667	8969.453333	252
std	86.746758	2.873001	1042.561303	1.109898	672.333705	21.377089	1.131543	1.456604	3753.531132	74
min	1.000000	1.000000	504.000000	0.000000	303.000000	1950.000000	0.000000	1.000000	2009.000000	82
25%	75.750000	3.000000	1392.250000	0.000000	903.000000	1967.000000	0.000000	2.000000	5996.250000	190
50%	150.500000	5.000000	2265.500000	1.000000	1502.000000	1986.000000	2.000000	3.000000	9031.000000	251
75%	225.250000	8.000000	3306.500000	2.000000	2129.500000	2004.250000	3.000000	4.000000	12316.000000	307
max	300.000000	10.000000	3998.000000	3.000000	2492.000000	2023.000000	3.000000	5.000000	14987.000000	435

```
freinds = {
    "stud_id" : [1,2,3,4,5],
    "name" : ['atharv','sandy','rudr','shiva','tejas'],
    "age":[19,20,21,22,23],
    "gender":['male','male','male','male','male'],
    "dept":['CSE(AI)','AIDS','CSE','CS(SE)','MECH'],
    "marks":[90,92,94,96,98],
    "attendace":[60,70,80,80,100],
    "internship":['nope','google','microsoft','JP morgan','infosys'],
}
```

```
df1=pd.DataFrame(freinds)
df1
```

	stud_id	name	age	gender	dept	marks	attendace	internship
0	1	atharv	19	male	CSE(AI)	90	60	nope
1	2	sandy	20	male	AIDS	92	70	google
2	3	rudr	21	male	CSE	94	80	microsoft
3	4	shiva	22	male	CS(SE)	96	80	JP morgan
4	5	tejas	23	male	MECH	98	100	infosys

```
data={
    "numbers" : [10,15,34,35,57,58,68,86,90]
}
df2=pd.DataFrame(data)
```

```
df.max()
df.min()
```

	0
Id	1
OverallQual	1
GrLivArea	504
GarageCars	0
TotalBsmtSF	303
YearBuilt	1950
FullBath	0
BedroomAbvGr	1
LotArea	2009
SalePrice	82494

dtype: int64

By using inbuilt function

```
df['SalePrice'].mean()
df2['numbers'].mean()
```

```
np.float64(50.333333333333336)
```

By using userdefined function

```
def mean(col):
  s=0
  n=0
  for x in col:
    s=s+x;
    n=n+1
  return s/n

print(mean(df['SalePrice']))
print(mean(df2['numbers']))
```

```
252262.90333333332
50.333333333333336
```

By using inbuilt funvtion

```
df['GarageCars'].var()
df2['numbers'].var()
```

```
829.75
```

```
def mean(col):
  val=sorted(df2['numbers'])
  n=len(val)
  mid=n//2
```

```
    if n%2==0:
      return (val[mid-1]+val[mid])/2
    else:
      return
```

By using userdefined function

```
def var(col):
  m=mean(col)
  s=0
  n=0
  for x in col:
    s=s+(x-m)**2
    n=n+1
  return s/n

print(var(df['GarageCars']))
```

By using inbuilt function

```
df['GarageCars'].std()
df2['numbers'].std()
```

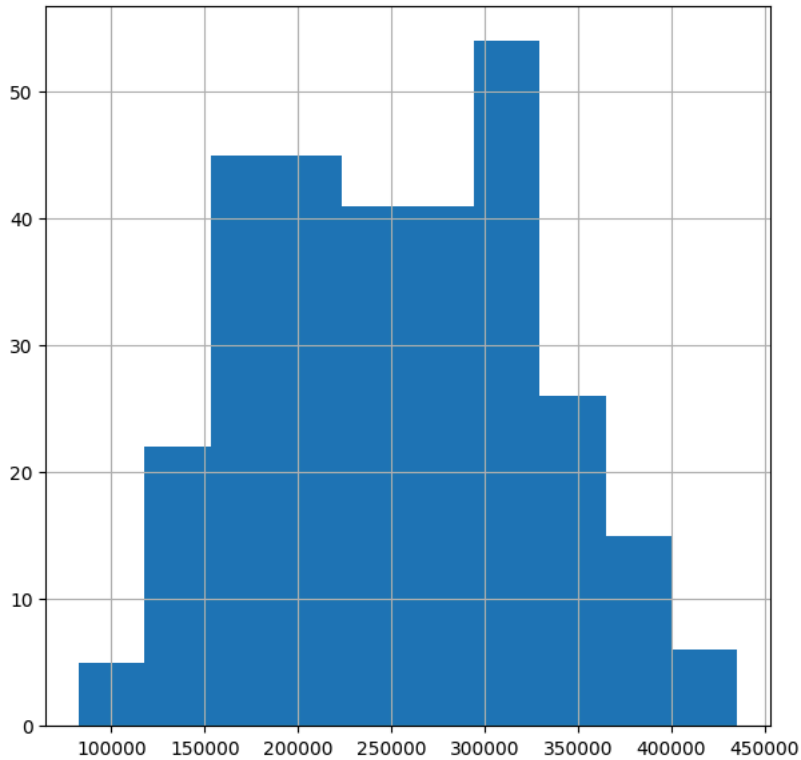
```
28.805381441668153
```

By using userdefined function

```
def std(col):
  return var(col) ** 0.5

print(std(df['GarageCars']))
```

```
df['SalePrice'].hist(bins=10,figsize=(7,7))
plt.show()
```



```
dummy = {
    "stud_id" : [1,2,3,4,5],
    "name" : ['atharv','sandy',pd.NA,'shiva','tejas'],
    "age":[19,20,pd.NA,22,23],
    "gender":['male','Female','male',pd.NA,'male'],
    "dept":['CSE(AI)','pd.NA','CSE','CS(SE)','MECH'],
    "marks":[90,92,94,96,98],
    "attendace":[60,70,pd.NA,80,100],
    "internship":['nope',pd.NA,'microsoft','JP morgan',pd.NA],
}
```

```
dummy_df=pd.DataFrame(dummy)
dummy_df
```

	stud_id	name	age	gender	dept	marks	attendace	internship
0	1	atharv	19	male	CSE(AI)	90	60	nope
1	2	sandy	20	Female	pd.NA	92	70	<NA>
2	3	<NA>	<NA>	male	CSE	94	<NA>	microsoft
3	4	shiva	22	<NA>	CS(SE)	96	80	JP morgan
4	5	tejas	23	male	MECH	98	100	<NA>

Detect missing values

```
dummy_df.isnull().sum()
```

	0
stud_id	0
name	1
age	1
gender	1
dept	0
marks	0
attendace	1
internship	2

dtype: int64

handle the missing values A] drop rows with missing values

```
dummy_df.dropna()
```

	stud_id	name	age	gender	dept	marks	attendace	internship
0	1	atharv	19	male	CSE(AI)	90	60	nope

B]fill the missing values in data

```
for col in dummy_df.columns:
  if dummy_df[col].dtype!='object':
    dummy_df[col]=dummy_df[col].fillna(dummy_df[col].mean())

dummy_df
```

	stud_id	name	age	gender	dept	marks	attendace	internship
0	1	atharv	19	male	CSE(AI)	90	60	nope
1	2	sandy	20	Female	pd.NA	92	70	<NA>
2	3	<NA>	<NA>	male	CSE	94	<NA>	microsoft
3	4	shiva	22	<NA>	CS(SE)	96	80	JP morgan
4	5	tejas	23	male	MECH	98	100	<NA>

```
dummy_df.fillna(0)
```

```
/tmp/ipython-input-3708968863.py:1: FutureWarning: Downcasting object dtype arrays on .fillna, .ffill, .bfill is deprecated
  dummy_df.fillna(0)
```

	stud_id	name	age	gender	dept	marks	attendace	internship
0	1	atharv	19	male	CSE(AI)	90	60	nope
1	2	sandy	20	Female	pd.NA	92	70	0
2	3	0	0	male	CSE	94	0	microsoft
3	4	shiva	22	0	CS(SE)	96	80	JP morgan
4	5	tejas	23	male	MECH	98	100	0

```
dummy_df['gender']=dummy_df['gender'].map({'male':0,'Female':1})
dummy_df
```

	stud_id	name	age	gender	dept	marks	attendace	internship
0	1	atharv	19	0.0	CSE(AI)	90	60	nope
1	2	sandy	20	1.0	pd.NA	92	70	<NA>
2	3	<NA>	<NA>	0.0	CSE	94	<NA>	microsoft
3	4	shiva	22	NaN	CS(SE)	96	80	JP morgan
4	5	tejas	23	0.0	MECH	98	100	<NA>

```
dummy_df=pd.get_dummies(dummy_df,columns=['dept'])
dummy_df
```

	stud_id	name	age	gender	marks	attendace	internship	dept_CS(SE)	dept_CSE	dept_CSE(AI)	dept_MECH	dept_pd.NA
0	1	atharv	19	0.0	90	60	nope	False	False	True	False	False
1	2	sandy	20	1.0	92	70	<NA>	False	False	False	False	True
2	3	<NA>	<NA>	0.0	94	<NA>	microsoft	False	True	False	False	False
3	4	shiva	22	NaN	96	80	JP morgan	True	False	False	False	False
4	5	tejas	23	0.0	98	100	<NA>	False	False	False	True	False

Vertical Data Integration (Row-wise) Combines datasets with same structure by stacking rows

```
marks=pd.DataFrame({
    "stud_id" : [1,2,3,4,5],
    "marks":[90,92,94,96,98],
})

attendance=pd.DataFrame({
    "stud_id" : [1,2,3,4,5],
    "attendace":[60,70,80,90,100]
})

df3=pd.concat([marks,attendance],axis=1)
print(df3)
```

```
   stud_id  marks  stud_id  attendace
0        1     90        1         60
1        2     92        2         70
2        3     94        3         80
3        4     96        4         90
4        5     98        5        100
```

Horizontal Data Integration (Column-wise) Combines different attributes for the same entities. Student Academic & Attendance Data Using concat(axis=1)

```
df3=pd.concat([marks,attendance],axis=0)
print(df3)
```

```
   stud_id  marks  attendace
0        1   90.0        NaN
1        2   92.0        NaN
2        3   94.0        NaN
3        4   96.0        NaN
4        5   98.0        NaN
0        1    NaN       60.0
1        2    NaN       70.0
```

```
2        3    NaN       80.0
3        4    NaN       90.0
4        5    NaN      100.0
```

```
students=pd.DataFrame({
    'stu_id':[1,2,3,4,5],
    'name':['atharv','sandy','rudr','shiva','tejas'],
})
result=pd.DataFrame({
    'stu_id':[1,2,3,5],
    'marks':[20,30,45,45]
})


df4=pd.merge(students,result,on='stu_id',how='inner')
print(df4)
```

```
   stu_id    name  marks
0       1  atharv     20
1       2   sandy     30
2       3    rudr     45
3       5   tejas     45
```

Key-based Data Integration (Joins) Using merge() Integrates data based on a common key.

```
df4=pd.merge(students,result,on='stu_id',how='left')
print(df4)
```

```
   stu_id    name  marks
0       1  atharv   20.0
1       2   sandy   30.0
2       3    rudr   45.0
3       4   shiva    NaN
4       5   tejas   45.0
```

```
df4=pd.merge(students,result,on='stu_id',how='right')
print(df4)
```

```
   stu_id    name  marks
0       1  atharv     20
1       2   sandy     30
2       3    rudr     45
3       5   tejas     45
```

```
df4=pd.merge(students,result,on='stu_id',how='outer')
print(df4)
```

```
   stu_id    name  marks
0       1  atharv   20.0
1       2   sandy   30.0
2       3    rudr   45.0
3       4   shiva    NaN
4       5   tejas   45.0
```

integration using index using join() joins datasets using index instead of columns

```
df1=students.set_index('stu_id')
df2=result.set_index('stu_id')
integrated_data=df1.join(df2)
print(integrated_data)
```

```
          name  marks
stu_id
1       atharv   20.0
2        sandy   30.0
3         rudr   45.0
4        shiva    NaN
5        tejas   45.0
```

Data transformation

MinMaxScaler vs StandardScaler Both are feature scaling techniques, but they work differently.

A. MinMaxScaler

Purpose: Scales all numeric values to a fixed range, usually 0 to 1 (can be changed). Formula:

X scaled=$X-X$min/Xmax $-X$ min

X_min = minimum value of feature

X_max = maximum value of feature

```
from sklearn.preprocessing import MinMaxScaler
import pandas as pd

df = pd.DataFrame({'Age':[18,22,25,30,35]})
scaler = MinMaxScaler()
df_scaled = scaler.fit_transform(df)
print(df_scaled)
```

```
[[0.        ]
 [0.23529412]
 [0.41176471]
 [0.70588235]
 [1.        ]]
```

When to use:

Data is not normally distributed

You need fixed range for all features

Algorithms like Neural Networks, KNN, SVM are sensitive to scale

A. Scaling / Normalization / Standardization

Purpose: Make numerical features comparable.

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler
import pandas as pd

df = pd.DataFrame({'Age':[18,22,25,30], 'Salary':[25000,50000,60000,80000]})

scaler = MinMaxScaler()
df[['Age','Salary']] = scaler.fit_transform(df[['Age','Salary']])
print("Min-Max Scaled:\n", df)

scaler = StandardScaler()
df_scaled = scaler.fit_transform(df[['Age','Salary']])
print("Standardized:\n", df_scaled)
```

```
Min-Max Scaled:
         Age    Salary
0  0.000000  0.000000
1  0.333333  0.454545
2  0.583333  0.636364
3  1.000000  1.000000
Standardized:
 [[-1.31267933 -1.45174713]
 [-0.3995111  -0.18935832]
 [ 0.28536507  0.3155972 ]
 [ 1.42682536  1.32550825]]
```